5
Arithmetic and Algebra

The link between programming skills and mathematical ability has been well established. Indeed, the earliest computers were built by mathematicians to speed up their calculations. Pascal (who was a mathematician long before he became a programming language) built a gear-based mechanical adding machine in 1645. Such pioneering computer scientists as Turing and von Neumann had equal or even greater achievements in pure mathematics.

In this chapter we will explore programming challenges in arithmetic and algebra, which are ostensibly the most elementary parts of mathematics. That certain algorithms depend on advanced topics such as number theory proves that they are not as elementary as they seem.

5.1 Machine Arithmetic

Every programming language includes an integer data type supporting the four basic arithmetic operations: addition, subtraction, multiplication, and division. These operations are typically mapped almost directly to hardware-level arithmetic instructions, and so the size range of integers depends on the underlying processor.

Today's PCs are typically 32-bit machines, meaning that the standard integer data type supports integers roughly in the range $\pm 2^{31} = \pm$ 2,147,483,648. Thus we can safely count up to a billion or so with standard integers on conventional machines.

Most programming languages support `long` or even `long long` integer data types, which often define 64-bit or occasionally even 128-bit integers. Since $2^{63} =$ 9,223,372,036,854,775,808, we are talking several orders of magnitude beyond trillions.

This is a very big number, so big that just counting up to it at modern computer speeds will take far longer than you would be willing to wait. This is more than the number of pennies in the United States budget deficit, so it usually proves more than adequate except for mathematical research or programming contest challenges.

Conventional 32-bit integers are typically represented using four contiguous bytes, with 64-bit longs being arrays of eight bytes. This is inefficient when storing large numbers of not-so-very large numbers. For example, computer images are often represented as matrices of single-byte colors (i.e., 256 grey levels) for space efficiency.

Positive integers are represented as positive binary numbers. Negative numbers usually use a fancier representation such as twos-complement, which although more obscure makes it easier to do arithmetic in hardware.

Floating point numbers will be discussed in Section 5.5. The *magnitude* of numbers which can be represented as `float`s is astonishingly large, particularly when you use double-precision floating point numbers. Be aware, however, that this magnitude comes by representing the number in scientific notation, as $a \times 2^c$. Since a and c are both restricted to a given number of bits, there is still only a limited *precision*. Don't be fooled into thinking that `float`s give you the ability to count to very high numbers. Use integers and longs for such purposes.

5.1.1 Integer Libraries

Every programming language provides the basic arithmetic operators as primitives, and usually provides more advanced mathematical functions through standard libraries.

Make yourself aware of the integer functions in your math libraries. For example, the C/C++ library `stdlib.h` includes absolute value computation and random numbers, while `math.h` includes ceilings, floors, square roots, and exponentials.

The integer classes for C++ and Java are even more powerful. Indeed, the GNU g++ `Integer` class and the `java.math BigInteger` class both provide support for high-precision integers in excess of what we will develop from scratch in the next section. Use them when you can, but be aware that certain applications and special problems have attributes which require you to roll your own.

5.2 High-Precision Integers

Representing truly enormous integers requires stringing digits together. Two possible representations are —

- *Arrays of Digits* — The easiest representation for long integers is as an array of digits, where the initial element of the array represents the least significant digit. Maintaining a counter with the length of the number in digits can aid efficiency by minimizing operations which don't affect the outcome.
- *Linked Lists of Digits* — Dynamic structures are necessary if we are *really* going to do arbitrary precision arithmetic, i.e., if there is no upper bound on the length

of the numbers. Note, however, that 100,000-digit integers are pretty long by any standard, yet can be represented using arrays of only 100,000 bytes each. Such space is peanuts on today's machines.

In this section, we will implement the major arithmetic operations for the array-of-digits representation. Dynamic memory allocation and linked lists provide an illusion of being able to get unlimited amounts of memory on demand. However, linked structures can be wasteful of memory, since part of each node consists of links to other nodes.

What dynamic memory *really* provides is the freedom to use space where you need it. If you wanted to create a large array of high-precision integers, a few of which were large and most of which were small, *then* you would be far better off with a list-of-digits representation, since you can't afford to allocate enormous amounts of space for all of them.

Our bignum data type is represented as follows:

```
#define MAXDIGITS       100             /* maximum length bignum */

#define PLUS            1               /* positive sign bit */
#define MINUS           -1              /* negative sign bit */

typedef struct {
        char digits[MAXDIGITS];         /* represent the number */
        int signbit;                    /* PLUS or MINUS */
        int lastdigit;                  /* index of high-order digit */
} bignum;
```

Note that each digit (0–9) is represented using a single-byte character. Although it requires a little more care to manipulate such numbers, the space savings enables us to feel less guilty about not using linked structures. Assigning `1` and `-1` to be the possible values of `signbit` will prove convenient, because we can multiply signbits and get the right answer.

Note that there is no real reason why we have to do our computations in base-10. In fact, using higher numerical bases is more efficient, by reducing the number of digits needed to represent each number. Still, it eases the conversion to and from a nice printed representation:

```
print_bignum(bignum *n)
{
        int i;

        if (n->signbit == MINUS) printf("- ");
        for (i=n->lastdigit; i>=0; i--)
                printf("%c",'0'+ n->digits[i]);

        printf("\n");
}
```

For simplicity, our coding examples will ignore the possibility of overflow.

5.3 High-Precision Arithmetic

The first algorithms we learned in school were those for computing the four standard arithmetical operations: addition, subtraction, multiplication, and division. We learned to execute them without necessarily understanding the underlying theory.

Here we review these grade-school algorithms, with the emphasis on understanding why they work and how you can teach them to a computer. For all four operations, we interpret the arguments as $c = a \star b$, where $\star$ is $+$, $-$, $*$, or $/$.

- *Addition* — Adding two integers is done from right to left, with any overflow rippling to the next field as a carry. Allowing negative numbers complicates matters by turning addition into subtraction. This is best handled by reducing it to a special case:

```
add_bignum(bignum *a, bignum *b, bignum *c)
{
        int carry;                      /* carry digit */
        int i;                          /* counter */

        initialize_bignum(c);

        if (a->signbit == b->signbit) c->signbit = a->signbit;
        else {
                if (a->signbit == MINUS) {
                        a->signbit = PLUS;
                        subtract_bignum(b,a,c);
                        a->signbit = MINUS;
                } else {
                        b->signbit = PLUS;
                        subtract_bignum(a,b,c);
                        b->signbit = MINUS;
                }
                return;
        }

        c->lastdigit = max(a->lastdigit,b->lastdigit)+1;
        carry = 0;

        for (i=0; i<=(c->lastdigit); i++) {
                c->digits[i] = (char)
                        (carry+a->digits[i]+b->digits[i]) % 10;
                carry = (carry + a->digits[i] + b->digits[i]) / 10;
        }
```

```
        zero_justify(c);
}
```

Note a few things about the code. Manipulating the signbit is a non-trivial headache. We reduced certain cases to subtraction by negating the numbers and/or permuting the order of the operators, but took care to replace the signs first.

The actual addition is quite simple, and made simpler by initializing all the high-order digits to 0 and treating the final carry over as a special case of digit addition. The `zero_justify` operation adjusts `lastdigit` to avoid leading zeros. It is harmless to call after every operation, particularly as it corrects for -0:

```
zero_justify(bignum *n)
{
        while ((n->lastdigit > 0) && (n->digits[ n->lastdigit ]==0))
                n->lastdigit --;

        if ((n->lastdigit == 0) && (n->digits[0] == 0))
                n->signbit = PLUS;      /* hack to avoid -0 */
}
```

- *Subtraction* — Subtraction is trickier than addition because it requires borrowing. To ensure that borrowing terminates, it is best to make sure that the larger-magnitude number is on top.

```
subtract_bignum(bignum *a, bignum *b, bignum *c)
{
        int borrow;                     /* anything borrowed? */
        int v;                          /* placeholder digit */
        int i;                          /* counter */

        if ((a->signbit == MINUS) || (b->signbit == MINUS)) {
                b->signbit = -1 * b->signbit;
                add_bignum(a,b,c);
                b->signbit = -1 * b->signbit;
                return;
        }

        if (compare_bignum(a,b) == PLUS) {
                subtract_bignum(b,a,c);
                c->signbit = MINUS;
                return;
        }

        c->lastdigit = max(a->lastdigit,b->lastdigit);
        borrow = 0;
```

```
        for (i=0; i<=(c->lastdigit); i++) {
                v = (a->digits[i] - borrow - b->digits[i]);
                if (a->digits[i] > 0)
                        borrow = 0;
                if (v < 0) {
                        v = v + 10;
                        borrow = 1;
                }

                c->digits[i] = (char) v % 10;
        }

        zero_justify(c);
}
```

- *Comparison* — Deciding which of two numbers is larger requires a comparison operation. Comparison proceeds from highest-order digit to the right, starting with the sign bit:

```
compare_bignum(bignum *a, bignum *b)
{
        int i;                          /* counter */

        if ((a->signbit==MINUS) && (b->signbit==PLUS)) return(PLUS);
        if ((a->signbit==PLUS) && (b->signbit==MINUS)) return(MINUS);

        if (b->lastdigit > a->lastdigit) return (PLUS * a->signbit);
        if (a->lastdigit > b->lastdigit) return (MINUS * a->signbit);

        for (i = a->lastdigit; i>=0; i--) {
                if (a->digits[i] > b->digits[i])
                                return(MINUS * a->signbit);
                if (b->digits[i] > a->digits[i])
                                return(PLUS * a->signbit);
        }

        return(0);
}
```

- *Multiplication* — Multiplication seems like a more advanced operation than addition or subtraction. A people as sophisticated as the Romans had a difficult time multiplying, even though their numbers look impressive on building cornerstones and Super Bowls.

  The Roman's problem was that they did not use a radix (or base) number system. Certainly multiplication can be viewed as repeated addition and thus

solved in that manner, but it will be hopelessly slow. Squaring 999,999 by repeated addition requires on the order of a million operations, but is easily doable by hand using the row-by-row method we learned in school:

```
multiply_bignum(bignum *a, bignum *b, bignum *c)
{
        bignum row;                     /* represent shifted row */
        bignum tmp;                     /* placeholder bignum */
        int i,j;                        /* counters */

        initialize_bignum(c);

        row = *a;

        for (i=0; i<=b->lastdigit; i++) {
                for (j=1; j<=b->digits[i]; j++) {
                        add_bignum(c,&row,&tmp);
                        *c = tmp;
                }
                digit_shift(&row,1);
        }

        c->signbit = a->signbit * b->signbit;

        zero_justify(c);
}
```

Each operation involves shifting the first number one more place to the right and then adding the shifted first number d times to the total, where d is the appropriate digit of the second number. We might have gotten fancier than using repeated addition, but since the loop cannot spin more than nine times per digit, any possible time savings will be relatively small. Shifting a radix-number one place to the right is equivalent to multiplying it by the base of the radix, or 10 for decimal numbers:

```
digit_shift(bignum *n, int d)           /* multiply n by 10^d */
{
        int i;                          /* counter */

        if ((n->lastdigit == 0) && (n->digits[0] == 0)) return;

        for (i=n->lastdigit; i>=0; i--)
                n->digits[i+d] = n->digits[i];

        for (i=0; i<d; i++) n->digits[i] = 0;
```

```
        n->lastdigit = n->lastdigit + d;
}
```

- *Division* — Although long division is an operation feared by schoolchildren and computer architects, it too can be handled with a simpler core loop than might be imagined. Division by repeated subtraction is again far too slow to work with large numbers, but the basic repeated loop of shifting the remainder to the left, including the next digit, and subtracting off instances of the divisor is far easier to program than "guessing" each quotient digit as we were taught in school:

```
divide_bignum(bignum *a, bignum *b, bignum *c)
{
        bignum row;                     /* represent shifted row */
        bignum tmp;                     /* placeholder bignum */
        int asign, bsign;               /* temporary signs */
        int i,j;                        /* counters */

        initialize_bignum(c);

        c->signbit = a->signbit * b->signbit;

        asign = a->signbit;
        bsign = b->signbit;

        a->signbit = PLUS;
        b->signbit = PLUS;

        initialize_bignum(&row);
        initialize_bignum(&tmp);

        c->lastdigit = a->lastdigit;

        for (i=a->lastdigit; i>=0; i--) {
                digit_shift(&row,1);
                row.digits[0] = a->digits[i];
                c->digits[i] = 0;
                while (compare_bignum(&row,b) != PLUS) {
                        c->digits[i] ++;
                        subtract_bignum(&row,b,&tmp);
                        row = tmp;
                }
        }

        zero_justify(c);

        a->signbit = asign;
```

```
        b->signbit = bsign;
}
```

This routine performs integer division and throws away the remainder. If you want to compute the remainder of $a \div b$, you can always do $a - b(a \div b)$. Slicker methods will follow when we discuss modular arithmetic in Section 7.3. The correct sign for the quotient and remainder when one or more of the operators is negative is somewhat ill-defined, so don't be surprised if the answer varies with programming language.

- *Exponentiation* — Exponentiation is repeated multiplication, and hence subject to the same performance problems as repeated addition on large numbers. The trick is to observe that

$$a^n = a^{n \div 2} \times a^{n \div 2} \times a^{n \bmod 2}$$

so it can be done using only a logarithmic number of multiplications.

5.4 Numerical Bases and Conversion

The digit representation of a given radix-number is a function of which numerical *base* is used. Particularly interesting numerical bases include:

- *Binary* — Base-2 numbers are made up of the digits 0 and 1. They provide the integer representation used within computers, because these digits map naturally to on/off or high/low states.
- *Octal* — Base-8 numbers are useful as a shorthand to make it easier to read binary numbers, since the bits can be read off from the right in groups of three. Thus $10111001_2 = 371_8 = 249_{10}$. They also play a role in the only base-conversion joke ever written. Why do programmers think Christmas is Halloween? Because 31 Oct = 25 Dec!
- *Decimal* — We use base-10 numbers because we learned to count on our ten fingers. The ancient Mayan people used a base-20 number system, presumably because they counted on both fingers and toes.
- *Hexadecimal* — Base-16 numbers are an even easier shorthand to represent binary numbers, once you get over the fact that the digits representing 10 through 15 are "A" to "F."
- *Alphanumeric* — Occasionally, one sees even higher numerical bases. Base-36 numbers are the highest you can represent using the 10 numerical digits with the 26 letters of the alphabet. Any integer can be represented in base-X provided you can display X different symbols.

There are two distinct algorithms you can use to convert base-a number x to a base-b number y —

- *Left to Right* — Here, we find the most-significant digit of y first. It is the integer d_l such that

$$(d_l + 1)b^k > x \geq d_l b^k$$

where $1 \leq d_l \leq b - 1$. In principle, this can be found by trial and error, although you must to be able to compare the magnitude of numbers in different bases. This is analogous to the long-division algorithm described above.

- *Right to Left* — Here, we find the least-significant digit of y first. This is the remainder of x divided by b. Remainders are exactly what is computed when doing modular arithmetic in Section 7.3. The cute thing is that we can compute the remainder of x on a digit-by-digit basis, making it easy to work with large integers.

Right-to-left translation is similar to how we translated conventional integers to our bignum presentation. Taking the long integer mod 10 (using the `%` operator) enables us to peel off the low-order digit:

```
int_to_bignum(int s, bignum *n)
{
        int i;                          /* counter */
        int t;                          /* int to work with */

        if (s >= 0) n->signbit = PLUS;
        else n->signbit = MINUS;

        for (i=0; i<MAXDIGITS; i++) n->digits[i] = (char) 0;

        n->lastdigit = -1;

        t = abs(s);

        while (t > 0) {
                n->lastdigit ++;
                n->digits[ n->lastdigit ] = (t % 10);
                t = t / 10;
        }

        if (s == 0) n->lastdigit = 0;
}
```

Using a different modulus than 10 is the key to converting numbers to alternate bases.

5.5 Real Numbers

The branches of mathematics designed to work with real numbers are real important in understanding the real world. Newton had to develop calculus before he could develop the basic laws of motion. The need to integrate or solve systems of equations occurs in every area of science. The first computers were designed as number-crunching machines, and the numbers they were designed to crunch were real numbers.

Working with real numbers on computers is very challenging because floating point arithmetic has limited precision. The most important thing to remember about real numbers is that they are not *real* real numbers:

- Much of mathematics relies on the *continuity* of the reals, the fact that there always exists a number c between a and b if $a < b$. This is not true in real numbers as they are represented in a computer.

- Many algorithms rely on an assumption of *exact* computation. This is not true of real numbers as they are represented in a computer. The associativity of addition guarantees that

$$(a + b) + c = a + (b + c)$$

  Unfortunately, this is not necessarily true in computer arithmetic because of round-off errors.

There are several different types of numbers which we may well want to work with:

- *Integers* — These are the counting numbers, $-\infty, \ldots, -2, -1, 0, 1, 2, \ldots, \infty$. Subsets of the integers include the *natural* numbers (integers starting from 0) and the *positive* integers (those starting from 1), although the notation is not universal. A limiting aspect of integers is that there are gaps between them. An April Fool's Day edition of a newspaper once had a headline announcing, "Scientists Discover New Number Between 6 and 7." This is funny because while there is *always* a rational number between any two rationals x and y ($(x+y)/2$ is a good example), it would indeed be newsworthy if they found an *integer* between 6 and 7.

- *Rational Numbers* — These are the numbers which can be expressed as the ratio of two integers, i.e. c is rational if $c = a/b$ for integers a and b. Every integer can be represented by a rational, namely, $c/1$. The rational numbers are synonymous with fractions, provided we include *improper* fractions a/b where $a > b$.

- *Irrational Numbers* — There are many interesting numbers which are not rational numbers. Examples include $\pi = 3.1415926\ldots$, $\sqrt{2} = 1.41421\ldots$, and $e = 2.71828\ldots$. It can be proven that there does not exist any pair of integers x and y such that x/y equals any of these numbers.

  So how can you represent them on a computer? If you *really* need the values to arbitrary precision, they can be computed using Taylor series expansions. But for all practical purposes it suffices to approximate them using the ten digits or so.

5.5.1 Dealing With Real Numbers

The internal representation of floating point numbers varies from computer to computer, language to language, and compiler to compiler. This makes them a big pain to deal with.

There is an IEEE standard for floating point arithmetic which an increasing number of vendors adhere to, but you must always expect trouble on computations which require very high precision. Floating point numbers are represented in scientific notation, i.e., $a \times 2^c$, with a limited number of bits assigned to both the *mantissa* a and *exponent* c. Operating on two numbers with vastly different exponents often results in overflow or underflow errors, since the mantissa does not have enough bits to accommodate the answer.

Such issues are the source of many difficulties with roundoff errors. The most important problem occurs in testing for equality of real numbers, since there is usually enough garbage in the low-order bits of mantissa to render such tests meaningless. Never test whether a float is equal to zero, or any other float for that matter. Instead, test if it lies within an ϵ value plus or minus of the target.

Many problems will ask you to display an answer to a given number of digits of precision to the right of the decimal point. Here we must distinguish between *rounding* and *truncating*. Truncation is exemplified by the `floor` function, which converts a real number of an integer by chopping off the fractional part. Rounding is used to get a more accurate value for the least significant digit. To round a number X to k decimal digits, use the formula

$$\text{round}(X, k) = \text{floor}(10^k X + (1/2))/10^k$$

Use your language's formatted output function to display only the desired number of digits when so requested.

5.5.2 Fractions

Exact rational numbers x/y are best represented by pairs of integers x, y, where x is the *numerator* and y is the *denominator* of the fraction.

The basic arithmetic operations on rationals $c = x_1/y_1$ and $d = x_2/y_2$ are easy to program:

- *Addition* — We must find a common denominator before adding fractions, so

$$c + d = \frac{x_1 y_2 + x_2 y_1}{y_1 y_2}$$

- *Subtraction* — Same as addition, since $c - d = c + -1 \times d$, so

$$c - d = \frac{x_1 y_2 - x_2 y_1}{y_1 y_2}$$

- *Multiplication* — Since multiplication is repeated addition, it is easily shown that

$$c \times d = \frac{x_1 x_2}{y_1 y_2}$$

- *Division* — To divide fractions you multiply by the *reciprocal* of the denominator, so

$$c/d = \frac{x_1}{y_1} \times \frac{y_2}{x_2} = \frac{x_1 y_2}{x_2 y_1}$$

  But why does this work? Because under this definition, $d(c/d) = c$, which is exactly what we want division to mean.

Blindly implementing these operations leads to a significant danger of overflows. It is important to *reduce* fractions to their simplest representation, i.e., replace 2/4 by 1/2. The secret is to cancel out the *greatest common divisor* of the numerator and the denominator, i.e., the largest integer which divides both of these integers.

Finding the greatest common divisor by trial and error or exhaustive search can be very expensive. However, the Euclidean algorithm for gcd is efficient, very simple to program, and discussed in Section 7.2.1.

5.5.3 Decimals

The decimal representation of real numbers is just a special case of the rational numbers. A decimal number represents the sum of two numbers; the integer part to the left of the decimal point and the fractional part to the right of the decimal. Thus the fractional representation of the first five decimal digits of π is

$$3.1415 = (3/1) + (1415/10000) = 6283/2000$$

The denominator of the fractional part is 10^{i+1} if the rightmost non-zero digit lies i places to the right of the decimal point.

Converting a rational to a decimal number is easy, in principle; just divide the numerator by the denominator. The catch is that many fractions do not have a finite decimal representation. For example, $1/3 = 0.3333333\ldots$, and $1/7 = 0.14285714285714285714\ldots$. Usually a decimal representation with the first ten or so significant digits will suffice, but sometimes we want to know the exact representation, i.e., $1/30 = 0.0\overline{3}$ or $1/7 = 0.\overline{142857}$.

What fraction goes with a given repeating decimal number? We can find it by explicitly simulating the long division. The decimal expansion of fraction 1/7 is obtained by dividing 7 into $1.0000000\ldots$. The next digit of the quotient is obtained by multiplying the remainder by ten, adding the last digit (always zero), and finding how many times the denominator fits into this quantity. Notice that we get into an infinite loop the instant this quantity repeats. Thus the decimal digits between these positions repeats forever.

A simpler method results if we know (or guess) the length of the repeat. Suppose that the simple fraction a/b has a repeat R of length of l. Then $10^l(a/b) - (a/b) = R$, and

hence $a/b = R/(10^l - 1)$. To demonstrate, suppose we want the fraction associated with $a/b = 0.0123123\ldots$. The repeat length is three digits, and $R = 12.3$ by the formula above. Thus $a/b = 12.3/999 = 123/9990$.

5.6 Algebra

In its full glory, algebra is the study of groups and rings. High-school algebra is basically limited to the study of equations, defined over the operators addition and multiplication. The most important class of formulae are the *polynomials*, such that $P(x) = c_0 + c_1 x + c_2 x^2 + \ldots$, where x is the variable and c_i is the coefficient of the ith term x^i. The degree of a polynomial is the largest i such that c_i is non-zero.

5.6.1 Manipulating Polynomials

The most natural representation for an nth-degree univariate (one variable) polynomial is as an array of $n + 1$ coefficients c_0 through c_n. Such a representation makes short work of the basic arithmetic operations on polynomials:

- *Evaluation* — Computing $P(x)$ for some given x can easily be done by brute force, namely, computing each term $c_i x^n$ independently and adding them together. The trouble is that this will cost $O(n^2)$ multiplications where $O(n)$ suffice. The secret is to note that $x^i = x^{i-1}x$, so if we compute the terms from smallest degree to highest degree we can keep track of the current power of x, and get away with two multiplications per term ($x^{i-1} \times x$, and then $c_i \times x^i$).

  Alternately, one can employ *Horner's rule*, an even slicker way to do the same job:

$$a_n x^n + a_{n-1} x^{n-1} + \ldots + a_0 = ((a_n x + a_{n-1})x + \ldots)x + a_0$$

- *Addition/Subtraction* — Adding and subtracting polynomials is even easier than the same operations on long integers, since there is no borrowing or carrying. Simply add or subtract the coefficients of the ith terms for all i from zero to the maximum degree.

- *Multiplication* — The product of polynomials $P(x)$ and $Q(x)$ is the sum of the product of every pair of terms, where each term comes from a different polynomial:

$$P(x) \times Q(x) = \sum_{i=0}^{degree(P)} \sum_{j=0}^{degree(Q)} (c_i c_j) x^{i+j}$$

  Such an all-against-all operation is called a *convolution*. Other convolutions in this book include integer multiplication (all digits against all digits) and string matching (all possible positions of the pattern string against all possible text positions). There is an amazing algorithm (the fast Fourier transform, or FFT) which computes convolutions in $O(n \log n)$ time instead of $O(n^2)$, but it is well

beyond the scope of this book. Still, it is nice to recognize when you are doing a convolution so you know that such tools exist.

- *Division* — Dividing polynomials is a tricky business, since the polynomials are not closed under division. Note that $1/x$ may or may not be thought of as a polynomial, since it is x^{-1}, but $2x/(x^2+1)$ certainly isn't. It is a *rational function*.

Sometimes polynomials are *sparse*, meaning that for many coefficients $c_i = 0$. Sufficiently sparse polynomials should be represented as linked lists of coefficient/degree pairs. *Multivariate polynomials* are defined over more than one variable. The bivariate polynomial $f(x, y)$ can be represented by a matrix C of coefficients, such that $C[i][j]$ is the coefficient of $x^i y^j$.

5.6.2 Root Finding

Given a polynomial $P(x)$ and a target number t, the problem of *root finding* is identifying any or all x such that $P(x) = t$.

If $P(x)$ is a first-degree polynomial, the root is simply $x = (t - a_0)/a_1$, where a_i is the coefficient of x_i in $P(x)$. If $P(x)$ is a second-degree polynomial, then the *quadratic equation* applies:

$$x = \frac{-a_1 \pm \sqrt{{a_1}^2 - 4a_2(a_0 - t)}}{2a_2}$$

There are more complicated formulae for solving third- and fourth-degree polynomials before the good times end. No closed form exists for the roots of fifth-degree (quintic) or higher-degree equations.

Beyond quadratic equations, numerical methods are typically used. Any text on numerical analysis will describe a variety of root-finding algorithms, including Newton's method and Newton-Raphson, as well as many potential traps such as numerical stability. But the basic idea is that of binary search. Suppose a function $f(x)$ is *monotonically increasing* between l and u, meaning that $f(i) \leq f(j)$ for all $l \leq i \leq j \leq u$. Now suppose we want to find the x such that $f(x) = t$. We can compare $f((l + u)/2)$ with t. If $t < f((l + u)/2)$, then the root lies between l and $(l + u)/2$; if not, it lies between $(l+u)/2$ and u. We can keep recurring until the window is narrow enough for our taste.

This method can be used to compute square roots because this is equivalent to solving $x^2 = t$ between 1 and t for all $t \geq 1$. However, a simpler method to find the ith root of t uses exponential functions and logarithms to compute $t^{1/i}$.

5.7 Logarithms

You have probably noticed the `log` and `exp` buttons on your calculator, but quite likely have never used them. You may even have forgotten why they are there. A *logarithm* is simply an inverse exponential function. Saying $b^x = y$ is equivalent to saying that $x = \log_b y$.

The b term is known as the *base* of the logarithm. Two bases are of particular importance for mathematical and historical reasons. The *natural log*, usually denoted $\ln x$, is a base $e = 2.71828\ldots$ logarithm. The inverse of $\ln x$ is the exponential function $\exp x = e^x$. Thus by composing these functions we get

$$\exp(\ln x) = x$$

Less common today is the base-10 or *common logarithm*, usually denoted $\log x$. Common logarithms were particularly important in the days before pocket calculators.[1] Logarithms provided the easiest way to multiply big numbers by hand, either implicitly using a slide rule or explicitly by using a book of logarithms.

Logarithms are still useful for multiplication, particularly for exponentiation. Recall that $\log_a xy = \log_a x + \log_a y$; i.e., the log of a product is the sum of the logs. A direct consequence of this is that

$$\log_a n^b = b \cdot \log_a n$$

So how can we compute a^b for any a and b using the $\exp(x)$ and $\ln(x)$ functions? We know

$$a^b = \exp(\ln(a^b)) = \exp(b \ln a)$$

so the problem is reduced to one multiplication plus one call of each of these functions.

We can use this method to compute square roots since $\sqrt{x} = x^{1/2}$, and for any other fractional power as well. Such applications are one reason why the mathematics library of any reasonable programming language includes the ln and exp functions. Be aware that these are complicated numerical functions (computed using Taylor-series expansions) which have inherent computational uncertainty, so do not expect that $\exp(0.5 \ln 4)$ will give you exactly 2.

The other important fact to remember about logarithms is that it is easy to convert the logarithm from one base to another, as a consequence of the following formula:

$$\log_a b = \frac{\log_c b}{\log_c a}$$

Thus changing the base of $\log b$ from base-a to base-c simply involves dividing by $\log_c a$. Thus it is easy to write a common log function from a natural log function, and vice versa.

5.8 Real Mathematical Libraries

Math Libraries in C/C++

The standard C/C++ math library has several useful functions for working with real numbers:

[1] The authors of this book are old enough to remember this pre-1972 era.

```
#include <math.h>          /* include the math library */

double floor(double x);    /* chop off fractional part of x */
double ceil (double x);    /* raise x to next largest integer */
double fabs(double x);     /* compute the absolute value of x */

double sqrt(double x);     /* compute square roots */
double exp(double x);      /* compute e^x */
double log(double x);      /* compute the base-e logarithm */
double log10(double x);    /* compute the base-10 logarithm */
double pow(double x, double y);   /* compute x^y */
```

Math Libraries in Java

The java class *java.lang.Math* has all of these functions and a few more, most obviously a `round` function to take a real to the nearest integer.

5.9 Problems

5.9.1 Primary Arithmetic

PC/UVa IDs: 110501/10035, **Popularity:** A, **Success rate:** average **Level:** 1

Children are taught to add multi-digit numbers from right to left, one digit at a time. Many find the "carry" operation, where a 1 is carried from one digit position to the next, to be a significant challenge. Your job is to count the number of carry operations for each of a set of addition problems so that educators may assess their difficulty.

Input

Each line of input contains two unsigned integers less than 10 digits. The last line of input contains "`0 0`".

Output

For each line of input except the last, compute the number of carry operations that result from adding the two numbers and print them in the format shown below.

Sample Input

```
123 456
555 555
123 594
0 0
```

Sample Output

```
No carry operation.
3 carry operations.
1 carry operation.
```

5.9.2 Reverse and Add

PC/UVa IDs: 110502/10018, **Popularity:** A, **Success rate:** low **Level:** 1

The *reverse and add* function starts with a number, reverses its digits, and adds the reverse to the original. If the sum is not a palindrome (meaning it does not give the same number read from left to right and right to left), we repeat this procedure until it does.

For example, if we start with 195 as the initial number, we get 9,339 as the resulting palindrome after the fourth addition:

```
   195        786       1,473       5,214
   591        687       3,741       4,125
 +----     +----      +-----      +-----
   786      1,473       5,214       9,339
```

This method leads to palindromes in a few steps for almost all of the integers. But there are interesting exceptions. 196 is the first number for which no palindrome has been found. It has never been proven, however, that no such palindrome exists.

You must write a program that takes a given number and gives the resulting palindrome (if one exists) and the number of iterations/additions it took to find it.

You may assume that all the numbers used as test data will terminate in an answer with less than 1,000 iterations (additions), and yield a palindrome that is not greater than 4,294,967,295.

Input

The first line will contain an integer N ($0 < N \leq 100$), giving the number of test cases, while the next N lines each contain a single integer P whose palindrome you are to compute.

Output

For each of the N integers, print a line giving the minimum number of iterations to find the palindrome, a single space, and then the resulting palindrome itself.

Sample Input	*Sample Output*

`3`
`195`
`265`
`750`
	`4 9339`
	`5 45254`
	`3 6666`

5.9.3 The Archeologist's Dilemma

PC/UVa IDs: 110503/701, **Popularity:** A, **Success rate:** low **Level:** 1

An archaeologist, seeking proof of the presence of extraterrestrials in the Earth's past, has stumbled upon a partially destroyed wall containing strange chains of numbers. The left-hand part of these lines of digits is always intact, but unfortunately the right-hand one is often lost because of erosion of the stone. However, she notices that all the numbers with all its digits intact are powers of 2, so that the hypothesis that all of them are powers of 2 is obvious. To reinforce her belief, she selects a list of numbers on which it is apparent that the number of legible digits is strictly smaller than the number of lost ones, and asks you to find the smallest power of 2 (if any) whose first digits coincide with those of the list.

Thus you must write a program that, given an integer, determines the smallest exponent E (if it exists) such that the first digits of 2^E coincide with the integer (remember that more than half of the digits are missing).

Input

Each line contains a positive integer N not bigger than 2,147,483,648.

Output

For every one of these integers, print a line containing the smallest positive integer E such that the first digits of 2^E are precisely the digits of N, or, if there isn't one, the sentence "`no power of 2`".

Sample Input

```
1
2
10
```

Sample Output

```
7
8
20
```

5.9.4 Ones

PC/UVa IDs: 110504/10127, **Popularity:** A, **Success rate:** high **Level:** 2

Given any integer $0 \leq n \leq 10,000$ not divisible by 2 or 5, some multiple of n is a number which in decimal notation is a sequence of `1`'s. How many digits are in the smallest such multiple of n?

Input

A file of integers at one integer per line.

Output

Each output line gives the smallest integer $x > 0$ such that $p = \sum_{i=0}^{x-1} 1 \times 10^i$, where a is the corresponding input integer, $p = a \times b$, and b is an integer greater than zero.

Sample Input

```
3
7
9901
```

Sample Output

```
3
6
12
```

5.9.5 A Multiplication Game

PC/UVa IDs: 110505/847, **Popularity:** A, **Success rate:** high **Level:** 3

Stan and Ollie play the game of multiplication by multiplying an integer p by one of the numbers 2 to 9. Stan always starts with $p = 1$, does his multiplication, then Ollie multiplies the number, then Stan, and so on. Before a game starts, they draw an integer $1 < n < 4,294,967,295$ and the winner is whoever reaches $p \geq n$ first.

Input

Each input line contains a single integer n.

Output

For each line of input, output one line – either

```
Stan wins.
```

or

```
Ollie wins.
```

assuming that both of them play perfectly.

Sample Input

```
162
17
34012226
```

Sample Output

```
Stan wins.
Ollie wins.
Stan wins.
```

5.9.6 Polynomial Coefficients

PC/UVa IDs: 110506/10105, **Popularity:** B, **Success rate:** high **Level:** 1

This problem seeks the coefficients resulting from the expansion of the polynomial

$$P = (x_1 + x_2 + \ldots + x_k)^n$$

Input

The input will consist of a set of pairs of lines. The first line of the pair consists of two integers n and k separated with space ($0 < k, n < 13$). These integers define the power of the polynomial and the number of variables. The second line in each pair consists of k non-negative integers $n_1, \ldots, n_k$, where $n_1 + \ldots + n_k = n$.

Output

For each input pair of lines the output line should consist of one integer, the coefficient of the monomial $x_1^{n_1} x_2^{n_2} \ldots x_k^{n_k}$ in expansion of the polynomial $(x_1 + x_2 + \ldots + x_k)^n$.

Sample Input

```
2 2
1 1
2 12
1 0 0 0 0 0 0 0 0 0 1 0
```

Sample Output

```
2
2
```

5.9.7 The Stern-Brocot Number System

PC/UVa IDs: 110507/10077, **Popularity:** C, **Success rate:** high **Level:** 1

The *Stern-Brocot tree* is a beautiful way for constructing the set of all non-negative fractions $\frac{m}{n}$ where m and n are relatively prime. The idea is to start with two fractions $\left(\frac{0}{1}, \frac{1}{0}\right)$ and then repeat the following operation as many times as desired:

> Insert $\frac{m+m'}{n+n'}$ between two adjacent fractions $\frac{m}{n}$ and $\frac{m'}{n'}$.

For example, the first step gives us one new entry between $\frac{0}{1}$ and $\frac{1}{0}$,

$$\frac{0}{1}, \frac{1}{1}, \frac{1}{0}$$

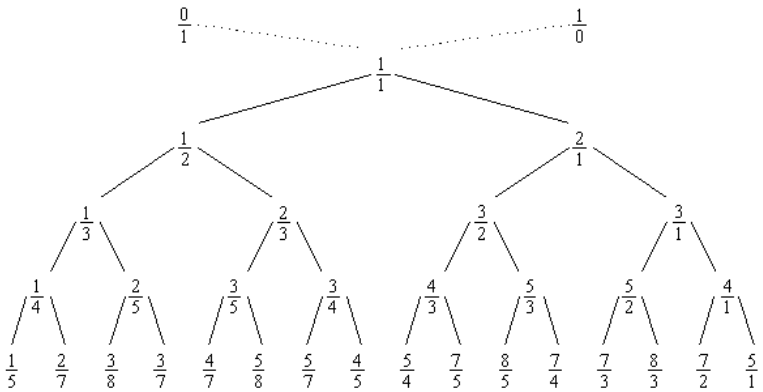
and the next gives two more:

$$\frac{0}{1}, \frac{1}{2}, \frac{1}{1}, \frac{2}{1}, \frac{1}{0}$$

The next gives four more:

$$\frac{0}{1}, \frac{1}{3}, \frac{1}{2}, \frac{2}{3}, \frac{1}{1}, \frac{3}{2}, \frac{2}{1}, \frac{3}{1}, \frac{1}{0}$$

The entire array can be regarded as an infinite binary tree structure whose top levels look like this–



This construction preserves order, and thus we cannot possibly get the same fraction in two different places.

We can, in fact, regard the *Stern-Brocot tree* as a *number system* for representing rational numbers, because each positive, reduced fraction occurs exactly once. Let us use the letters "`L`" and "`R`" to stand for going down the left or right branch as we proceed from the root of the tree to a particular fraction; then a string of `L`'s and `R`'s uniquely identifies a place in the tree. For example, `LRRL` means that we go left from $\frac{1}{1}$ down to $\frac{1}{2}$, then right to $\frac{2}{3}$, then right to $\frac{3}{4}$, then left to $\frac{5}{7}$. We can consider LRRL to be a representation of $\frac{5}{7}$. Every positive fraction gets represented in this way as a unique string of `L`'s and `R`'s.

Well, almost every fraction. The fraction $\frac{1}{1}$ corresponds to the empty string. We will denote it by I, since that looks something like 1 and stands for "identity."

In this problem, given a positive rational fraction, represent it in the *Stern-Brocot number system*.

Input

The input file contains multiple test cases. Each test case consists of a line containing two positive integers m and n, where m and n are relatively prime. The input terminates with a test case containing two `1`'s for m and n, and this case must not be processed.

Output

For each test case in the input file, output a line containing the representation of the given fraction in the *Stern-Brocot number system*.

Sample Input

```
5 7
878 323
1 1
```

Sample Output

```
LRRL
RRLRRLRLLLLRLRRR
```

5.9.8 Pairsumonious Numbers

PC/UVa IDs: 110508/10202, **Popularity:** B, **Success rate:** high **Level:** 4

Any set of n integers form $n(n-1)/2$ sums by adding every possible pair. Your task is to find the n integers given the set of sums.

Input

Each line of input contains n followed by $n(n-1)/2$ integer numbers separated by a space, where $2 < n < 10$.

Output

For each line of input, output one line containing n integers in non-descending order such that the input numbers are pairwise sums of the n numbers. If there is more than one solution, any one will do. If there is no solution, print "`Impossible`"...

Sample Input

```
3 1269 1160 1663
3 1 1 1
5 226 223 225 224 227 229 228 226 225 227
5 216 210 204 212 220 214 222 208 216 210
5 -1 0 -1 -2 1 0 -1 1 0 -1
5 79950 79936 79942 79962 79954 79972 79960 79968 79924 79932
```

Sample Output

```
383 777 886
Impossible
111 112 113 114 115
101 103 107 109 113
-1 -1 0 0 1
39953 39971 39979 39983 39989
```

5.10 Hints

5.1 Do we need to implement complete high-precision addition for this problem, or can we extract the number of carry operations using a simpler method?

5.3 Do we need to implement complete high-precision multiplication for this problem, or does the fact that we are looking for a power of 2 simplify matters?

5.4 Do we actually have to compute the number in order to find the number of digits it contains?

5.5 Might it be easier to solve a more general problem – who wins if they start with number x and end on number n?

5.6 Do we need to compute the resulting polynomial, or is there an easier way to calculate the resulting coefficient? Does the binomial theorem help?

5.8 Is an exhaustive search of the possibilities necessary? If so, look ahead to backtracking in Chapter 8.

5.11 Notes

5.2 A three-year computer search for an addition palindrome from 196 went up to 2 million digits without ever finding such a palindrome. It becomes progressively less likely that a palindrome exists the longer we search. See *http://www.fourmilab.ch/documents/threeyears/threeyears.html* for details.